



RESTAURANT MANAGEMENT SYSTEM

Point of Sale (POS) System

Database Design Document

V 2.0

CSC-271 — Database Systems

Namal University, Mianwali

By

Sr.	Name	Roll No
1	Muhammad Shuaib Noor	NUM-BSCS-2024-57
2	Allah Ditta	NUM-BSCS-2024-10
3	Shahbaz Ali	NUM-BSCS-2024-73

Department of Computer Sciences

Namal University

Mianwali, Pakistan

Submission Date: 14st June, 2026

REVISION HISTORY

Date	Version	Description	Approved By
21/06/2026	V 2.0	Added Chapter 3 — Logical Database Design (relational schema, functional dependencies, full 3NF normalization) and Chapter 4 — Implementation (language, database connectivity, GUI interfaces).	Instructor
15/05/2026	V 1.0	Initial submission covering project overview, entity identification, data dictionary, business rules, and the Entity Relationship Diagram.	Instructor

Contents

Revision History	1
1 Project Overview	4
1.1 Introduction	4
1.2 Problem Statement	5
1.3 Project Objectives	6
1.3.1 GitHub Repository Link	7
2 Conceptual Database Design	8
2.1 Entity	8
2.2 Data Dictionary	9
2.2.1 Branch	9
2.2.2 Employee	10
2.2.3 Salary	11
2.2.4 Shift	11
2.2.5 Category	11
2.2.6 Menu Item	12
2.2.7 Branch Menu Item	12
2.2.8 Deal Item	12
2.2.9 Branch Deal	13
2.2.10 Price	13
2.2.11 Order	14
2.2.12 Order Item	14
2.2.13 Inventory	15
2.2.14 Menu Ingredient	15
2.2.15 Discount	16
2.2.16 Deal	16
2.3 Business Rules	17
2.4 Entity Relationship Diagram	19
3 Logical Database Design	21
3.1 Relational Schema	21

3.1.1	Conversion Rules Applied	21
3.1.2	Entity Summary and Mapping	22
3.1.3	Final Relational Schemas	22
3.1.4	Relationship Mapping Summary	24
3.2	Functional Dependencies	25
3.3	Normalization	26
3.3.1	First Normal Form (1NF)	26
3.3.2	Second Normal Form (2NF)	26
3.3.3	Third Normal Form (3NF)	27
3.3.4	Final 3NF Schema	28
4	Implementation	30
4.1	Language / Framework	30
4.2	Database Connectivity	31
4.3	Interfaces	31
	References	36
	Appendix: AI Usage and Prompts	37

CHAPTER 1: PROJECT OVERVIEW

1.1. INTRODUCTION

Running a pizza restaurant with more than one branch is harder than it sounds. Every day, cashiers are taking orders, chefs are cooking, managers are checking stock, and admins are looking after staff schedules and pay — all at the same time, sometimes at three different locations. Without a proper system tying everything together, things slip through the cracks: the wrong order gets sent to the kitchen, an item runs out of stock mid-service, or a manager cannot figure out which cashier took a bad order.

This project builds a **Restaurant Management System** that acts as the digital backbone of a multi-branch pizza restaurant. At its core is a **Point of Sale (POS)** database that keeps track of everything: orders, menu items, pricing history, promotional deals, staff, shifts, salaries, and branch-level inventory. The system is designed so that three types of employees — **Admin**, **Receptionist**, and **Kitchen Staff** — can all work on it at the same time without stepping on each other's toes.

The main things the system handles are:

- Taking and recording customer orders through the receptionist's screen.
- Automatically handing orders over to whichever kitchen staff member has the least work at that moment.
- Watching inventory levels in real time and sending a low-stock warning before anything runs out.
- Keeping a full history of menu prices so that old receipts always show the correct amount the customer actually paid.
- Letting each branch decide independently which menu items and deals it offers, without changing what the other branches show.
- Recording every staff member's working shifts and salary payments so that managers have a complete audit trail.

The database is implemented in **MySQL 9.7** and the GUI is built with **C++ and Raylib**. The application is developed using **Visual Studio Code**.

1.2. PROBLEM STATEMENT

Many small restaurant chains still manage their day-to-day operations using paper records, spreadsheets, or simple cash registers that do not talk to each other. This creates a whole set of headaches that hurt both the staff and the customers.

- **No real-time stock tracking.** Staff have no way of knowing how much of an ingredient is actually left. A customer orders a pizza, the cashier accepts the order, and only then does the kitchen discover there is no dough left. This wastes the customer's time and damages the restaurant's reputation.
- **Unfair order assignment.** Without automation, orders end up being given to whichever chef is nearest or easiest to reach, not the one who actually has the capacity. Some chefs get swamped while others stand around — and service slows down for everyone.
- **Lost price history.** When management raises the price of a pizza from Rs. 850 to Rs. 950, the old orders in the system may suddenly show the new price instead of what the customer actually paid. This creates accounting errors and customer disputes.
- **No accountability.** If an order arrives at the table completely wrong, nobody can tell whether the cashier took it down incorrectly or the kitchen prepared it wrong. Without that information, the same mistake will keep happening.
- **Customers cannot change their mind.** Once an order is placed verbally or on a slip of paper, there is no clean way to modify or cancel it before cooking starts. The customer has to argue at the counter, and the staff has to cross things out manually.
- **Branches operate in isolation.** Each branch keeps its own paper records. A manager at head office cannot look at all three branches at once. Comparing sales, stock, or staff performance across locations requires manually collecting and combining data from separate stores.
- **Menu changes affect everyone at once.** If the Lahore branch runs out of one topping and needs to remove an item from the menu for a day, there is no way to do that without removing it from every other branch too.

All of these problems add up to lost revenue, unhappy customers, and frustrated staff. A centralised, properly designed database system solves every one of them.

1.3. PROJECT OBJECTIVES

General Objective: To design and implement a reliable, secure, and easy-to-use Point of Sale system for a multi-branch pizza restaurant that automates order handling, tracks inventory in real time, creates clear accountability for every transaction, and gives management a complete picture of operations across all branches.

Specific Objectives:

- **Automate order assignment.** The system automatically routes each new order to the kitchen staff member who currently has the fewest active orders, so workload stays balanced and no chef gets overwhelmed.
- **Block orders when stock is too low.** Before an order is confirmed, the system checks whether enough inventory is available at that branch. If not, the order cannot be placed and the receptionist is told exactly what is short.
- **Keep a full price history.** Every time a menu item's price changes, the old price is archived rather than overwritten. This means any past order always shows the price the customer actually paid, not today's price.
- **Track who did what.** Every order records both the receptionist who took it and the kitchen staff member who cooked it. If something goes wrong, management can immediately see who was responsible at each step.
- **Allow order changes before cooking starts.** A customer can modify or cancel an order as long as its status is still "Pending." Once the kitchen marks it "In Preparation," changes are locked out automatically.
- **Support multiple branches independently.** Each branch manages its own inventory, its own staff, and its own list of available menu items and deals — all within the same shared system.
- **Send low-stock warnings automatically.** Whenever any inventory item drops below its reorder level, a real-time notification is sent to the receptionist, kitchen staff, and admin so they can act before stock actually runs out.
- **Record staff attendance and shifts.** Every clock-in and clock-out is logged so that admins always know who worked, when, and for how long.
- **Manage salaries securely.** All salary payment records are stored in the system and are only visible to employees with Admin access — receptionists and kitchen staff cannot see this information at all.

- **Support combo deals branch by branch.** Managers can create promotional deals and decide which branches offer them. If a deal includes an item that is out of stock at one branch, that entire deal is automatically hidden at that branch.

1.3.1. GitHub Repository Link

<https://github.com/muhammadshoaibnoor/Database-System-Project.git>

CHAPTER 2: CONCEPTUAL DATABASE DESIGN

2.1. ENTITY

The table below lists all core entities in the system. Associative (junction) tables such as BRANCH_MENU_ITEM, DEAL_ITEM, and MENU_INGREDIENT are not listed here; they appear as part of the relational schema in Chapter 3.

Sr.	Entity Name	Description
01	Branch	A physical restaurant location. It has its own address, phone number, opening/closing hours, and a flag that shows whether it is currently open for business.
02	Employee	Any member of staff who logs into the system. An employee belongs to exactly one branch and has a role: Admin, Receptionist, or Kitchen Staff.
03	Salary	A record of a single salary payment made to a staff member. Keeping separate records lets the system store full payment history rather than just a current figure.
04	Shift	A record of one working session for a staff member, including the date, start time, and end time.
05	Category	A group that menu items belong to, for example “Pizza”, “Cold Drinks”, or “Add-Ons”.
06	Menu Item	A product that the restaurant sells. Every menu item belongs to a category and has a type (Pizza, AddOn, or Coldrink).
07	Price	A historical price record for one menu item, with the date the price started and the date it ended. This lets the system remember what a customer was charged on any past order.
08	Order	A complete customer transaction. It records which branch processed it, who took it, who cooked it, whether a discount was applied, the total amount, and the current status.

09	Order Item	A single line inside an order, representing either one menu item or one deal being ordered, along with how many and the unit price at that moment.
10	Inventory	A record of one raw material stocked at a specific branch — for example, “Flour” at the Lahore branch. It tracks current quantity, unit, cost, and the reorder threshold.
11	Menu Ingredient	A recipe entry that links a menu item to an inventory item and says how much of that ingredient is used to make one unit of that menu item.
12	Discount	A named discount offer with a value and a validity period. A discount can be applied to an order at the time of sale.
13	Deal	A promotional combo that bundles multiple menu items together and sells them at a single fixed price.

2.2. DATA DICTIONARY

This section describes every attribute of each entity. Foreign keys are not listed here — they appear in the Relational Schema (Section 3.1).

2.2.1. Branch

Sr.	Name	Data Type	Constraint	Description
01	BranchId	INT	PK, NOT NULL	A unique number that identifies each branch.
02	BranchName	VARCHAR(100)	NOT NULL, UNIQUE	The public name of the branch, e.g. “Lahore Branch”.
03	Address	TEXT	NOT NULL	The full street address of the branch.
04	PhoneNumber	VARCHAR(20)	NOT NULL	The branch contact phone number.
05	Email	VARCHAR(100)	UNIQUE	An optional email address for the branch.
06	OpeningTime	TIME	NOT NULL	The time the branch opens every day.

07	ClosingTime	TIME	NOT NULL	The time the branch closes every day.
08	IsActive	BOOLEAN	DEFAULT TRUE	Set to FALSE when a branch is closed permanently or temporarily.

2.2.2. Employee

Sr.	Name	Data Type	Constraint	Description
01	EmployeeId	VARCHAR(50)	PK, NOT NULL	The unique ID each staff member uses to log in.
02	FirstName	VARCHAR(50)	NOT NULL	The staff member's first name.
03	LastName	VARCHAR(50)	NOT NULL	The staff member's last name.
04	Email	VARCHAR(100)	NOT NULL, UNIQUE	Work email address.
05	PhoneNumber	VARCHAR(20)	NOT NULL	Staff contact number.
06	PasswordHash	VARCHAR(255)	NOT NULL	A bcrypt hash of the staff member's password. Plain-text passwords are never stored.
07	Role	VARCHAR(20)	NOT NULL	One of: Admin, Receptionist, Kitchen Staff.
08	IsActive	BOOLEAN	DEFAULT TRUE	FALSE when a staff member has left the company.
09	HireDate	DATE	NOT NULL	The date the staff member was hired.
10	ResignationDate	DATE	NULL	The date the staff member left. NULL means they are still employed.

2.2.3. Salary

Sr.	Name	Data Type	Constraint	Description
01	SalaryId	INT	PK, NOT NULL	A unique number for each salary payment record.
02	Amount	DECIMAL(10,2)	NOT NULL	The amount paid in Pakistani Rupees.
03	PaymentDate	DATE	NOT NULL	The date the payment was made.
04	Notes	TEXT	NULL	Any extra information about the payment, such as a bonus reason.

2.2.4. Shift

Sr.	Name	Data Type	Constraint	Description
01	ShiftId	INT	PK, NOT NULL	A unique number for each shift record.
02	ShiftDate	DATE	NOT NULL	The calendar date of the shift.
03	StartTime	TIME	NOT NULL	The time the staff member clocked in.
04	EndTime	TIME	NULL	The time the staff member clocked out. NULL if the shift is still ongoing.
05	ShiftType	VARCHAR(20)	NOT NULL	One of: Morning, Evening, Night.

2.2.5. Category

Sr.	Name	Data Type	Constraint	Description
01	CategoryId	INT	PK, NOT NULL	A unique number for each category.
02	CategoryName	VARCHAR(50)	NOT NULL, UNIQUE	The name of the category, e.g. "Pizza" or "Cold Drinks".

2.2.6. Menu Item

Sr.	Name	Data Type	Constraint	Description
01	MenuItemId	INT	PK, NOT NULL	A unique number for each menu item.
02	ItemName	VARCHAR(100)	NOT NULL	The name shown on the menu, e.g. “Chicken Tikka Pizza”.
03	Description	TEXT	NULL	A short description of what the item is.
04	MenuItemType	VARCHAR(20)	NOT NULL	One of: Pizza, AddOn, Coldrink.

2.2.7. Branch Menu Item

Sr.	Name	Data Type	Constraint	Description
01	BranchMenuItemId	INT	PK, NOT NULL	A unique number for each branch–menu-item assignment.
02	BranchId	INT	FK, NOT NULL	The branch this availability record belongs to.
03	MenuItemId	INT	FK, NOT NULL	The menu item whose availability is being recorded.
04	IsAvailable	BOOLEAN	DEFAULT TRUE	FALSE when this item is temporarily unavailable at this branch only.

2.2.8. Deal Item

Sr.	Name	Data Type	Constraint	Description
01	DealItemId	INT	PK, NOT NULL	A unique number for each deal–item line.
02	DealId	INT	FK, NOT NULL	The deal this item belongs to.
03	MenuItemId	INT	FK, NOT NULL	The menu item included in this deal.

04	Quantity	INT	NOT NULL, > 0	How many units of this menu item are included in the deal.
----	----------	-----	------------------	--

2.2.9. Branch Deal

Sr.	Name	Data Type	Constraint	Description
01	BranchDealId	INT	PK, NOT NULL	A unique number for each branch–deal assignment.
02	BranchId	INT	FK, NOT NULL	The branch this deal availability record belongs to.
03	DealId	INT	FK, NOT NULL	The deal whose availability is being recorded.
04	IsAvailable	BOOLEAN	DEFAULT TRUE	FALSE when this deal is hidden at this branch, for example because one of its items is out of stock.

2.2.10. Price

Sr.	Name	Data Type	Constraint	Description
01	PriceId	INT	PK, NOT NULL	A unique number for each price record.
02	Amount	DECIMAL(10,2)	NOT NULL	The price in Pakistani Rupees that was charged during this period.
03	EffectiveDate	DATE	NOT NULL	The date from which this price applies.
04	EndDate	DATE	NULL	The last date this price was valid. NULL means it is the current price.

2.2.11. Order

Sr.	Name	Data Type	Constraint	Description
01	OrderId	INT	PK, NOT NULL	A unique number for each order.
02	OrderDateTime	DATETIME	NOT NULL	The exact date and time when the order was placed.
03	Status	VARCHAR(20)	NOT NULL	The current stage of the order: Pending, In Preparation, Completed, or Cancelled.
04	PaymentStatus	VARCHAR(20)	NOT NULL	Whether the order has been paid: Unpaid, Paid, or Refunded.
05	CashTendered	DECIMAL(10,2)	NULL	The amount of cash the customer handed over.
06	TotalAmount	DECIMAL(10,2)	NOT NULL	The final amount due after any discount is applied.

2.2.12. Order Item

Sr.	Name	Data Type	Constraint	Description
01	OrderItemId	INT	PK, NOT NULL	A unique number for each line item within an order.
02	Quantity	INT	NOT NULL, > 0	How many of this item the customer ordered.
03	UnitPrice	DECIMAL(10,2)	NOT NULL	The price per unit at the exact time the order was placed. Stored so that future price changes do not affect this record.

2.2.13. Inventory

Sr.	Name	Data Type	Constraint	Description
01	InventoryId	INT	PK, NOT NULL	A unique number for each stock record.
02	InventoryName	VARCHAR(100)	NOT NULL	The name of the raw material, e.g. "Mozzarella Cheese".
03	Quantity	DECIMAL(10,2)	NOT NULL, DEFAULT 0	How much stock is currently available.
04	Unit	VARCHAR(20)	NOT NULL	The unit of measurement, e.g. kg, grams, or pieces.
05	ReorderLevel	DECIMAL(10,2)	NOT NULL	When stock falls to this level, the system sends a warning.
06	UnitCost	DECIMAL(10,2)	NULL	How much it costs to buy one unit of this ingredient.
07	LastRestocked	DATE	NULL	The date the stock was most recently topped up.

2.2.14. Menu Ingredient

Sr.	Name	Data Type	Constraint	Description
01	MenuIngredientId	INT	PK, NOT NULL	A unique number for each recipe entry.
02	RequiredQuantity	DECIMAL(10,2)	NOT NULL	How much of the inventory item is consumed when one unit of the menu item is made.
03	Unit	VARCHAR(20)	NOT NULL	The unit for the required quantity, e.g. grams.

2.2.15. Discount

Sr.	Name	Data Type	Constraint	Description
01	DiscountId	INT	PK, NOT NULL	A unique number for each discount.
02	DiscountName	VARCHAR(100)	NOT NULL	A short name for the discount, e.g. “Ramadan Special”.
03	DiscountValue	DECIMAL(10,2)	NOT NULL	The size of the discount, either a flat rupee amount or a percentage.
04	StartDate	DATETIME	NOT NULL	The date and time from which this discount can be used.
05	EndDate	DATETIME	NULL	The date and time after which the discount expires. NULL means it is still active.
06	IsActive	BOOLEAN	DEFAULT TRUE	A quick flag to enable or disable a discount without deleting it.

2.2.16. Deal

Sr.	Name	Data Type	Constraint	Description
01	DealId	INT	PK, NOT NULL	A unique number for each deal.
02	DealName	VARCHAR(100)	NOT NULL	The name of the deal, e.g. “Family Feast”.
03	TotalAmount	DECIMAL(10,2)	NOT NULL	The single fixed price a customer pays for the whole combo.
04	IsActive	BOOLEAN	DEFAULT TRUE	Whether the deal is currently being sold.

2.3. BUSINESS RULES

The following rules define exactly how the system behaves. Each rule has a direct impact on how data is stored and validated.

Branch and Employee Rules

- A branch can employ many staff members, but a newly created branch may temporarily have no staff assigned yet.
- Every employee belongs to exactly one branch — you cannot be assigned to two branches at the same time.
- An Admin can manage many other employees within the system.
- Every employee must have one of three roles: Admin, Receptionist, or Kitchen Staff.
- Different branches can offer different menu items. For example, the Islamabad branch may sell a dish that the Mianwali branch does not.
- If a branch runs out of an ingredient, the affected menu item is marked unavailable *at that branch only* — it stays available at all other branches.
- Only an Admin can view or update the Salary table. Receptionists and Kitchen Staff have no access to salary data at all.

Shift Rules

- An employee can have many shift records over time — one per working day.
- A new hire may have zero shift records until their first working day.
- Each shift record belongs to exactly one employee.
- A shift records the date, clock-in time, and clock-out time. If the staff member is still working, EndTime is left NULL.

Salary Rules

- Every salary payment creates a new record rather than overwriting an old one, so the full payment history is always available.
- Each payment record is linked to exactly one employee.
- Only Admin employees can read or create salary records.

Menu and Category Rules

- Every menu item must belong to exactly one category.
- A category can contain many menu items.
- Menu items are classified as one of: Pizza, AddOn, or Coldrink.
- Every time a menu item's price changes, a new Price record is created with the new amount and start date. The old record's EndDate is set to yesterday so it is never lost.
- At any given moment, exactly one Price record for each menu item should have a NULL EndDate — that is the current price.

Order Rules

- Every order must be taken by exactly one receptionist and prepared by exactly one kitchen staff member.
- An order can contain many order items; each order item belongs to exactly one order.
- While an order is still “Pending”, a customer can change their mind: items can be added, removed, or the whole order can be cancelled.
- Once a kitchen staff member moves an order to “In Preparation”, it is locked. No more changes or cancellations are allowed.
- A completed refund changes PaymentStatus to “Refunded”.
- If the wrong order was placed, the receptionist who took it is accountable. If the food was cooked incorrectly, the kitchen staff member who prepared it is accountable. The system records both names on every order.

Order Assignment Rules

- When a new order comes in, the system automatically finds the kitchen staff member at that branch with the fewest active (non-completed, non-cancelled) orders and assigns the new order to them.
- If two or more kitchen staff members have the same number of active orders, the system picks the one whose EmployeeId comes first alphabetically.

Discount Rules

- A maximum of one discount can be applied to any single order. An order may also have no discount at all.
- A discount can only be applied if its IsActive flag is TRUE and the current date falls between its StartDate and EndDate.

Inventory Rules

- Each inventory record belongs to exactly one branch. The same raw material (e.g. Mozzarella) will have separate inventory records for each branch so that stock levels can differ.
- An order cannot be confirmed if placing it would reduce any ingredient's quantity below zero at the branch.
- When stock falls at or below the ReorderLevel, the system sends a real-time alert to all active employees at that branch.

Deal Rules

- A deal must contain at least one menu item, and a menu item can be included in many different deals.
- When a customer orders a deal, every ingredient for every item inside that deal is deducted from the branch's inventory.
- Customers can mix deals and single items within the same order.
- If any menu item inside a deal is currently unavailable at a branch, the entire deal is automatically hidden from that branch's menu.

2.4. ENTITY RELATIONSHIP DIAGRAM

Figure 3.1.2 shows the complete ERD for the Restaurant POS System. It uses Crow's Foot notation and covers all 16 tables — 13 core entities plus 3 junction tables (BRANCH_MENU_ITEM, MENU_INGREDIENT, and BRANCH_DEAL). Every entity shows its primary key, attributes, and all relationships with their cardinalities.

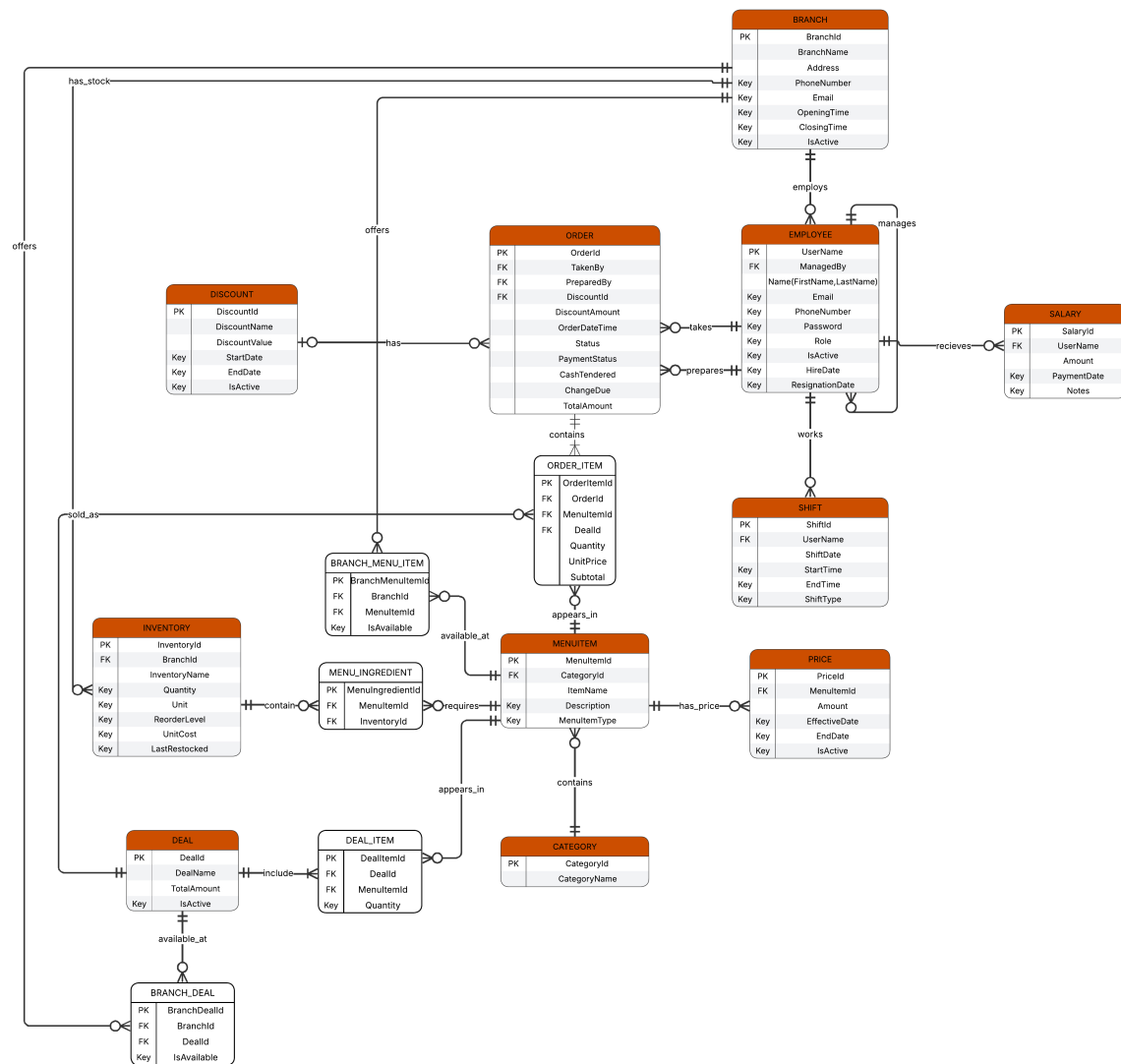


Figure 2.1: Entity Relation Ship Diagram

CHAPTER 3: LOGICAL DATABASE DESIGN

3.1. RELATIONAL SCHEMA

3.1.1. Conversion Rules Applied

To turn the ERD into actual database tables, the following standard mapping rules were followed:

- Rule 1. Strong entity** — becomes its own table. Its identifying attribute becomes the primary key.
- Rule 2. Weak entity** — becomes its own table. Its primary key is a combination of its own partial key and the primary key of the owner entity (which is also a foreign key).
- Rule 3. One-to-Many (1:N) relationship** — the primary key of the “one” side is added as a foreign key column in the “many” side table.
- Rule 4. Many-to-Many (M:N) relationship** — a new junction (bridge) table is created. Its primary key is the combination of both entities’ primary keys, and both are also foreign keys.
- Rule 5. One-to-One (1:1) relationship** — the primary key of one entity is added as a foreign key in the other, depending on which side has total participation.
- Rule 6. Multi-valued attribute** — becomes a separate table containing the host entity’s primary key plus the attribute value.

3.1.2. Entity Summary and Mapping

The ERD has 13 core entities. Three M:N relationships generate additional junction tables, giving a total of **16 relations**.

#	Table Name	Role in the System
1	BRANCH	Stores information about each restaurant location.
2	EMPLOYEE	Stores employee profiles (renamed from USER to avoid SQL conflicts).
3	SALARY	Records each individual salary payment made to an employee.
4	SHIFT	Records each working shift for an employee.
5	ORDER	Records every customer order placed at any branch.
6	ORDER_ITEM	Records each individual item or deal line within an order.
7	MENUITEM	Lists all products the restaurant sells.
8	CATEGORY	Groups menu items into logical categories.
9	PRICE	Stores the full pricing history of every menu item.
10	DEAL	Defines promotional combo offers.
11	DEAL_ITEM	Links each deal to the menu items it contains.
12	INVENTORY	Tracks raw material stock at each branch.
13	DISCOUNT	Defines available discount offers.
14	BRANCH_MENU-ITEM	Junction table — controls which menu items are available at each branch.
15	MENU_INGREDIENT	Junction table — defines the recipe (ingredients) for each menu item.
16	BRANCH_DEAL	Junction table — controls which deals are available at each branch.

3.1.3. Final Relational Schemas

Primary keys are underlined. Foreign keys are *italicised*.

BRANCH (BranchId, BranchName, Address, PhoneNumber, Email, OpeningTime, ClosingTime, IsActive)

EMPLOYEE (EmployeeId, *ManagedBy*, *BranchId*, FirstName, LastName, Email, PhoneNumber, PasswordHash, Role, IsActive, HireDate, ResignationDate)

ManagedBy → EMPLOYE(EmployeeId) | *BranchId* → BRANCH(BranchId)

SALARY (SalaryId, *EmployeeId*, Amount, PaymentDate, Notes)

EmployeeId → EMPLOY(*EmployeeId*)

SHIFT (ShiftId, *EmployeeId*, ShiftDate, StartTime, EndTime, ShiftType)

EmployeeId → EMPLOY(*EmployeeId*)

DISCOUNT (DiscountId, DiscountName, DiscountValue, StartDate, EndDate, IsActive)

ORDER (OrderId, *BranchId*, *TakenBy*, *PreparedBy*, *DiscountId*, OrderDateTime, Status, PaymentStatus, CashTendered, TotalAmount)

BranchId → BRANCH | *TakenBy*, *PreparedBy* → EMPLOY(*EmployeeId*) | *DiscountId* → DISCOUNT

CATEGORY (CategoryId, CategoryName)

MENUITEM (MenuItemId, *CategoryId*, ItemName, Description, MenuItemType)

CategoryId → CATEGORY(*CategoryId*)

PRICE (PriceId, *MenuItemId*, Amount, EffectiveDate, EndDate)

MenuItemId → MENUITEM(*MenuItemId*)

INVENTORY (InventoryId, *BranchId*, InventoryName, Quantity, Unit, ReorderLevel, UnitCost, LastRestocked)

BranchId → BRANCH(*BranchId*)

DEAL (DealId, DealName, TotalAmount, IsActive)

ORDER_ITEM (OrderItemId, *OrderId*, *MenuItemId*, *DealId*, Quantity, UnitPrice)

OrderId → ORDER | *MenuItemId* → MENUITEM | *DealId* → DEAL

DEAL_ITEM (DealItemId, *DealId*, *MenuItemId*, Quantity)

DealId → DEAL | *MenuItemId* → MENUITEM

BRANCH_MENU_ITEM (BranchMenuItemId, BranchId, MenuItemId, IsAvailable)

BranchId → BRANCH | MenuItemId → MENUITEM | UNIQUE(BranchId, MenuItemId)

MENU_INGREDIENT (MenuIngredientId, MenuItemId, InventoryId, RequiredQuantity, Unit)

MenuItemId → MENUITEM | InventoryId → INVENTORY

BRANCH_DEAL (BranchDealId, BranchId, DealId, IsAvailable)

BranchId → BRANCH | DealId → DEAL | UNIQUE(BranchId, DealId)

3.1.4. Relationship Mapping Summary

#	Relationship	Type	Between	Rule	How It Was Mapped
1	employs	1:N	BRANCH → EMPLOY	Rule 3	BranchId added as FK in EMPLOY
2	manages	1:N	EMPLOY → EMPLOY	Rule 3	ManagedBy self-referencing FK
3	receives	1:N	EMPLOY → SALARY	Rule 3	EmployeeId FK in SALARY
4	works	1:N	EMPLOY → SHIFT	Rule 3	EmployeeId FK in SHIFT
5	takes	1:N	EMPLOY → ORDER	Rule 3	TakenBy FK in ORDER
6	prepares	1:N	EMPLOY → ORDER	Rule 3	PreparedBy FK in ORDER
7	offers	1:N	DISCOUNT → ORDER	Rule 3	DiscountId FK in ORDER
8	contains	1:N	ORDER → ORDER_ITEM	Rule 3	OrderId FK in ORDER_ITEM
9	sold as	M:N	ORDER_ITEM, MENUITEM	Rule 4	MenuItemId FK in ORDER_ITEM
10	belongs to	1:N	CATEGORY → MENUITEM	Rule 3	CategoryId FK in MENUITEM
11	has price	1:N	MENUITEM → PRICE	Rule 3	MenuItemId FK in PRICE

12	available at	M:N	BRANCH, MENUITEM	Rule 4	BRANCH_MENU_ITEM junction table
13	requires	M:N	MENUITEM, INVENTORY	Rule 4	MENU_INGREDIENT junction table
14	has stock	1:N	BRANCH → INVENTORY	Rule 3	BranchId FK in INVENTORY
15	includes	M:N	DEAL, MENUITEM	Rule 4	DEAL_ITEM junction table
16	appears in	M:N	DEAL, ORDER_ITEM	Rule 4	DealId FK in ORDER_ITEM
17	offered at	M:N	BRANCH, DEAL	Rule 4	BRANCH_DEAL junction table

3.2. FUNCTIONAL DEPENDENCIES

A functional dependency $A \rightarrow B$ means that knowing the value of A is enough to uniquely determine the value of B. Below are the complete, minimal sets of non-trivial functional dependencies for all 16 relations.

- **BRANCH:** BranchId $\rightarrow$ BranchName, Address, PhoneNumber, Email, OpeningTime, ClosingTime, IsActive
- **EMPLOY:** EmployeeId $\rightarrow$ ManagedBy, BranchId, FirstName, LastName, Email, PhoneNumber, PasswordHash, Role, IsActive, HireDate, ResignationDate
- **SALARY:** SalaryId $\rightarrow$ EmployeeId, Amount, PaymentDate, Notes
- **SHIFT:** ShiftId $\rightarrow$ EmployeeId, ShiftDate, StartTime, EndTime, ShiftType
- **DISCOUNT:** DiscountId $\rightarrow$ DiscountName, DiscountValue, StartDate, EndDate, IsActive
- **ORDER:** OrderId $\rightarrow$ BranchId, TakenBy, PreparedBy, DiscountId, OrderDateTime, Status, PaymentStatus, CashTendered, TotalAmount
- **CATEGORY:** CategoryId $\rightarrow$ CategoryName
- **MENUITEM:** MenuItemId $\rightarrow$ CategoryId, ItemName, Description, MenuItemType
- **PRICE:** PriceId $\rightarrow$ MenuItemId, Amount, EffectiveDate, EndDate
- **INVENTORY:** InventoryId $\rightarrow$ BranchId, InventoryName, Quantity, Unit, ReorderLevel, UnitCost, LastRestocked

- **DEAL:** DealId → DealName, TotalAmount, IsActive
- **ORDER_ITEM:** OrderItemId → OrderId, MenuItemId, DealId, Quantity, UnitPrice
- **DEAL_ITEM:** DealItemId → DealId, MenuItemId, Quantity
- **BRANCH_MENU_ITEM:** BranchMenuItemId → BranchId, MenuItemId, IsAvailable. Also: {BranchId, MenuItemId} → IsAvailable (natural composite key enforced via UNIQUE constraint).
- **MENU_INGREDIENT:** MenuIngredientId → MenuItemId, InventoryId, RequiredQuantity, Unit
- **BRANCH_DEAL:** BranchDealId → BranchId, DealId, IsAvailable. Also: {BranchId, DealId} → IsAvailable (natural composite key enforced via UNIQUE constraint).

3.3. NORMALIZATION

3.3.1. First Normal Form (1NF)

A table is in 1NF when every cell holds exactly one value — no lists, no repeating groups, no composite values packed into a single column.

When the original ERD was designed, the EMPLOYEE entity had a composite attribute called *Name* that bundled *FirstName* and *LastName* together as one field. That was split into two separate columns in the EMPLOY table.

Repeating groups also appeared naturally in the design — for example, a menu item can have many prices over its lifetime. Rather than cramming multiple prices into one column (which would break 1NF), a separate PRICE table was created where each row holds one price record for one menu item.

After these adjustments, all 16 tables are in 1NF.

3.3.2. Second Normal Form (2NF)

A table is in 2NF when it is already in 1NF and every non-key column depends on the *whole* primary key, not just part of it. This only becomes a concern when a primary key is made up of two or more columns.

In this schema, every table uses a single-column primary key (either a surrogate integer like *SalaryId*, or a natural key like *EmployeeId*). Because the primary key is always a single column, there is no such thing as a “partial dependency” — a non-key column cannot depend on only part of a one-column key.

All 16 tables are therefore automatically in 2NF.

3.3.3. Third Normal Form (3NF)

A table is in 3NF when it is in 2NF and no non-key column depends on another non-key column. In other words, every non-key column must tell us something directly about the primary key — not something about another non-key column.

During the design review, the following problems were found and fixed:

Problem Found	Normal Form Violated	Why It Was a Problem	How It Was Fixed
EMPLOYEE.Name held FirstName and LastName as one composite value.	1NF	A single column should not hold two separate pieces of information.	Split into two columns: FirstName and LastName.
ORDER had a DiscountAmount column.	3NF	The discount amount can always be worked out from the discount record and the order total. Storing it created a risk of the two values getting out of sync.	Removed. The discount amount is now calculated when needed.
ORDER had a ChangeDue column.	Derived attribute	ChangeDue is simply CashTendered minus TotalAmount. There is no reason to store a value that can always be recalculated.	Removed. Calculated in the application whenever a receipt is printed.
ORDER_ITEM had a Subtotal column.	Derived attribute	Subtotal is always Quantity multiplied by UnitPrice. Storing it risked inconsistency.	Removed. Calculated on the fly in queries.
ORDER was missing a BranchId column.	Structural gap	Without knowing which branch processed an order, branch-level reports are impossible.	BranchId added as a NOT NULL foreign key in ORDER.

PRICE had an IsActive column.	3NF	Whether a price is active can always be determined from the EndDate column (NULL means active). Storing IsActive as a separate column creates a transitive dependency.	Removed. Active price is found using: EndDate IS NULL OR EndDate $\geq$ CURRENT_DATE.
MENU_INGREDIENT had no quantity column.	Incomplete design	Without knowing how much of each ingredient each menu item uses, the system cannot accurately deduct stock when an order is placed.	Added RequiredQuantity and Unit columns.
EMPLOY stored a plain-text Password.	Security violation	Storing readable passwords is a serious security risk.	Replaced with PasswordHash, which stores a bcrypt hash.

After applying all the corrections above, every one of the 16 tables satisfies **Third Normal Form**. There are no transitive dependencies remaining.

3.3.4. Final 3NF Schema

The complete set of 16 normalised relations is presented below. Primary keys are underlined; foreign keys are *italicised*.

BRANCH (BranchId, BranchName, Address, PhoneNumber, Email, OpeningTime, ClosingTime, IsActive)

EMPLOY (EmployeeId, *ManagedBy*, *BranchId*, FirstName, LastName, Email, PhoneNumber, PasswordHash, Role, IsActive, HireDate, ResignationDate)

SALARY (SalaryId, *EmployeeId*, Amount, PaymentDate, Notes)

SHIFT (ShiftId, *EmployeeId*, ShiftDate, StartTime, EndTime, ShiftType)

DISCOUNT (DiscountId, DiscountName, DiscountValue, StartDate, EndDate, IsActive)

ORDER (OrderId, *BranchId*, *TakenBy*, *PreparedBy*, *DiscountId*, OrderDateTime, Status, PaymentStatus, CashTendered, TotalAmount)

CATEGORY (CategoryId, CategoryName)

MENUITEM (MenuItemId, *CategoryId*, ItemName, Description, MenuItemType)

PRICE (PriceId, *MenuItemId*, Amount, EffectiveDate, EndDate)

INVENTORY (InventoryId, *BranchId*, InventoryName, Quantity, Unit, ReorderLevel, UnitCost, LastRestocked)

DEAL (DealId, DealName, TotalAmount, IsActive)

ORDER_ITEM (OrderItemId, *OrderId*, *MenuItemId*, *DealId*, Quantity, UnitPrice)

DEAL_ITEM (DealItemId, *DealId*, *MenuItemId*, Quantity)

BRANCH_MENU_ITEM (BranchMenuItemId, *BranchId*, *MenuItemId*, IsAvailable)

MENU_INGREDIENT (MenuIngredientId, *MenuItemId*, *InventoryId*, RequiredQuantity, Unit)

BRANCH_DEAL (BranchDealId, *BranchId*, *DealId*, IsAvailable)

CHAPTER 4: IMPLEMENTATION

4.1. LANGUAGE / FRAMEWORK

The database engine is **MySQL 9.7**. It was chosen because it is free, widely used in real-world desktop applications, has strong support for structured data management, and provides reliable performance for multi-branch restaurant operations.

The desktop GUI is built in **C++** using the **Raylib** library. Raylib is a simple, easy-to-use library for building graphical applications. It was chosen because:

- It provides a straightforward window and input API without a large framework overhead.
- C++ gives full control over memory and performance.
- The team is learning C++ in current coursework, making it the natural choice for practical implementation.
- Raylib requires no external dependencies beyond its own headers and is easy to set up inside Visual Studio Code.

The entire application is developed and compiled using **Visual Studio Code** with the MinGW-w64 compiler toolchain on Windows.

The overall architecture has three distinct layers:

1. **Presentation layer** — the Raylib GUI that the employee sees and interacts with.
2. **Business logic layer** — C++ functions that enforce all the rules (stock checks, auto-assignment of orders, etc.).
3. **Data layer** — the MySQL 9.7 tables that physically store the data.

4.2. DATABASE CONNECTIVITY

The C++ application connects to MySQL 9.7 using the **MySQL Connector/C** library. The connector files were obtained directly from the MySQL Server installation folder:

- The `include` folder from the MySQL Server directory was copied into the project and added to the compiler's include path, so that `#include <mysql.h>` resolves correctly.
- `libmysql.dll` was copied into the project's output directory so that the executable can find it at runtime.
- `libmysql.lib` was added to the linker settings in `tasks.json` inside Visual Studio Code so that the build links against the MySQL client library.

4.3. INTERFACES

The GUI is split into role-based screens. Each employee logs in with their Employee ID and password, and the application shows only the screens their role permits.

Login Screen

When the application starts, the employee sees a login form rendered with Raylib. It contains an Employee ID input field, a password field (characters masked), and a Login button. Credentials are verified against the EMPLOY table. Three consecutive failed attempts temporarily lock the account.

Order Entry Screen (Receptionist)

This is the main screen for receptionists. The left panel lists all available menu items and deals for the current branch. The receptionist clicks items to add them to the current order shown on the right. A discount can be selected from a dropdown and the total updates immediately. Clicking "Place Order" confirms the transaction and deducts inventory. If any item is out of stock, a red warning panel appears and the order cannot be submitted.

Kitchen Display Screen (Kitchen Staff)

Kitchen staff see a scrollable list of orders assigned to them, with the oldest order at the top. Each entry shows the order ID, list of items, and the time it was placed. Buttons allow the staff to advance an order from "Pending" to "In Preparation" and then to "Completed". Completed orders are removed from the list automatically.

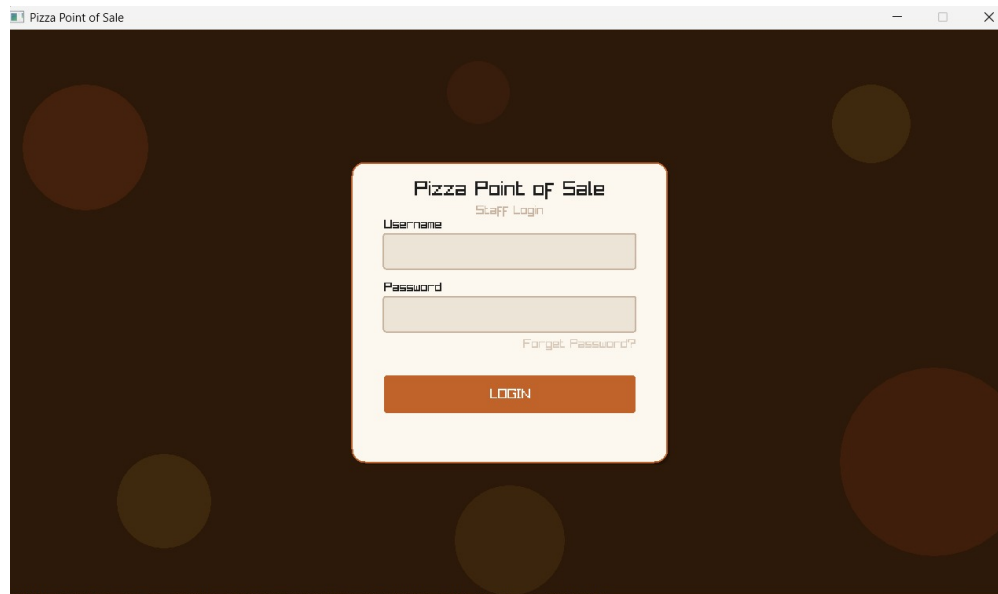


Figure 4.2: Login Screen

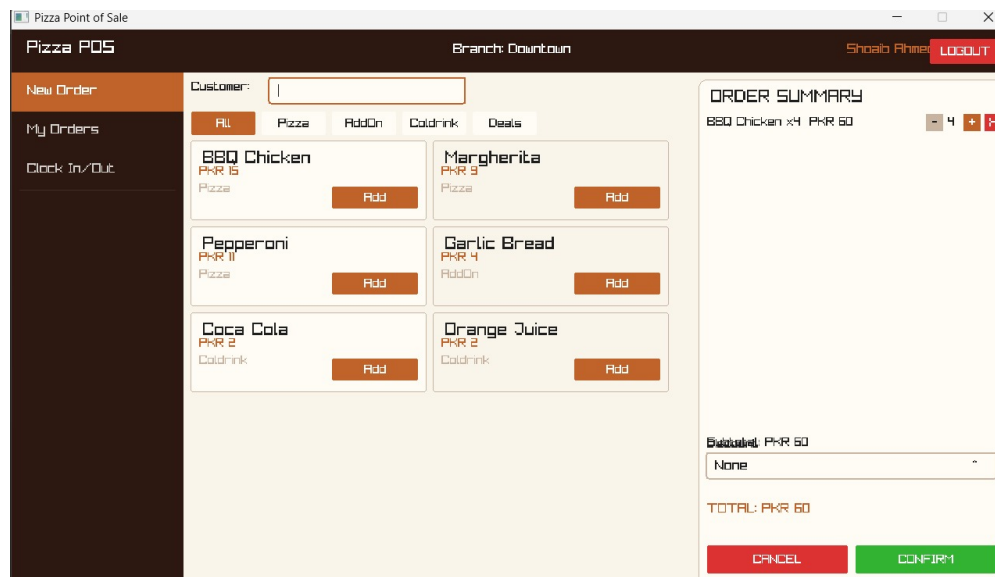


Figure 4.3: Order Entry Screen – Receptionist View

Inventory Management Panel (Admin)

Admins see a table of all inventory items at their branch. Quantities at or below the reorder level are highlighted in red. From this panel, admins can add stock and adjust the reorder level for any item.

Employee Management Panel (Admin)

A full CRUD interface for managing staff. Admins can add a new employee, update details, or mark an employee as inactive when they leave. A “Record Salary Payment” button opens a dialog and saves a new record to the SALARY table.

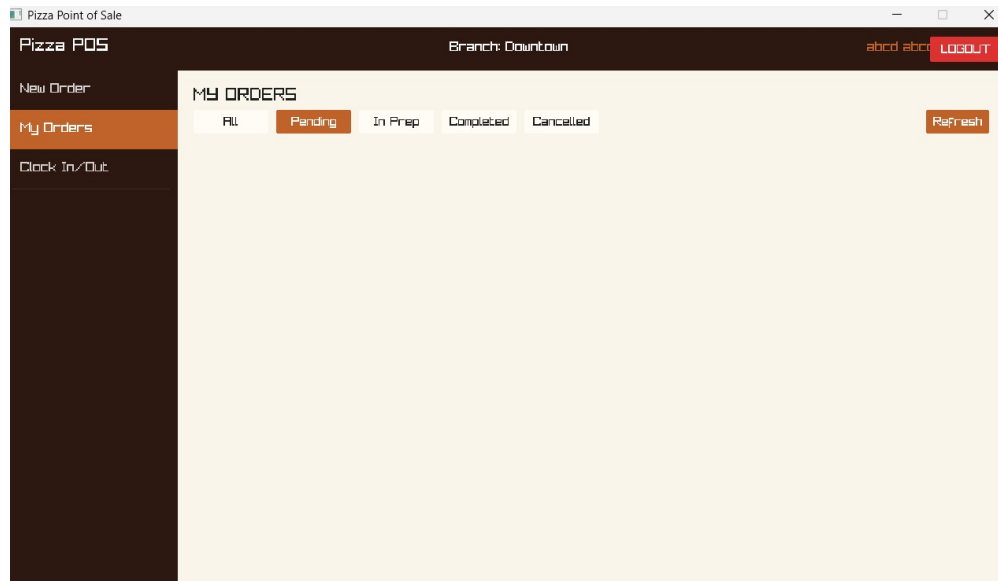


Figure 4.4: Kitchen Display Screen – Kitchen Staff View

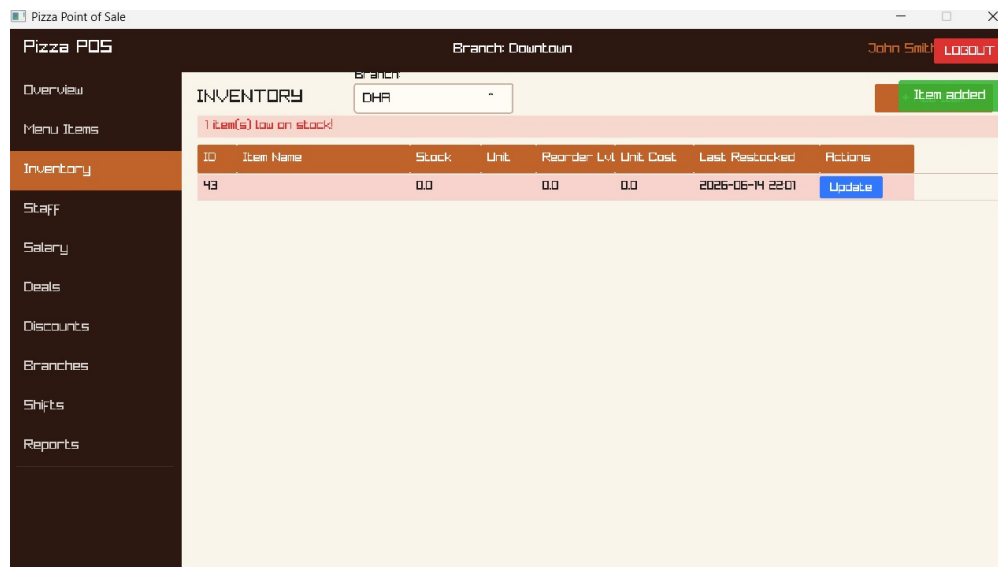


Figure 4.5: Inventory Management Panel – Admin View

Shift Scheduling Panel (Admin)

A calendar-style view where admins log shifts for any staff member, entering the date, shift type (Morning/Evening/Night), start time, and end time. All records are stored in the SHIFT table.

Sales Report Screen (Admin)

Admins select a branch, a start date, and an end date, then click “Generate Report”. The screen queries the ORDER and ORDER_ITEM tables and displays a summary table of results with total orders and total revenue for the selected period.

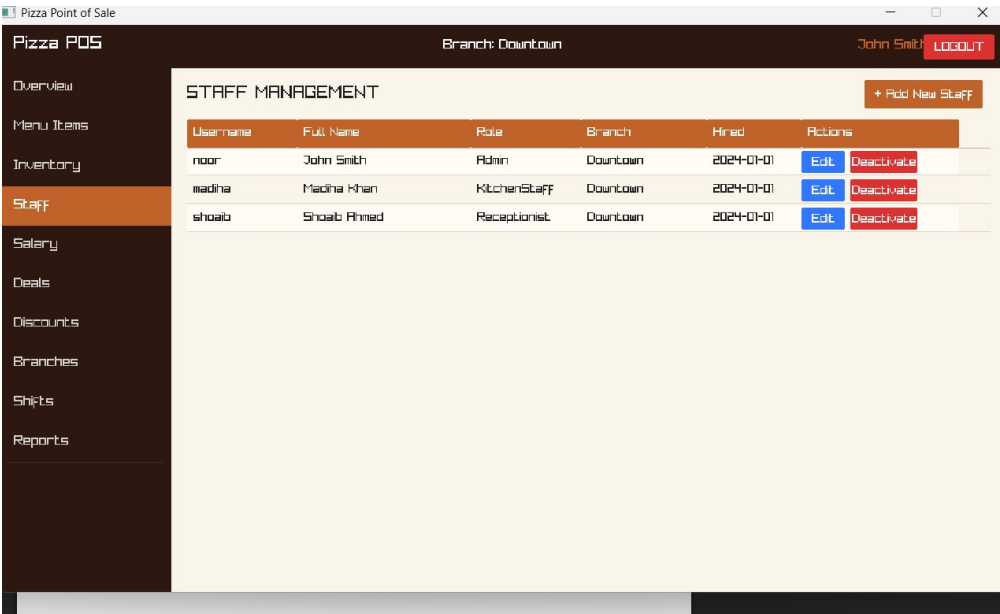


Figure 4.6: Employee Management Panel – Admin View

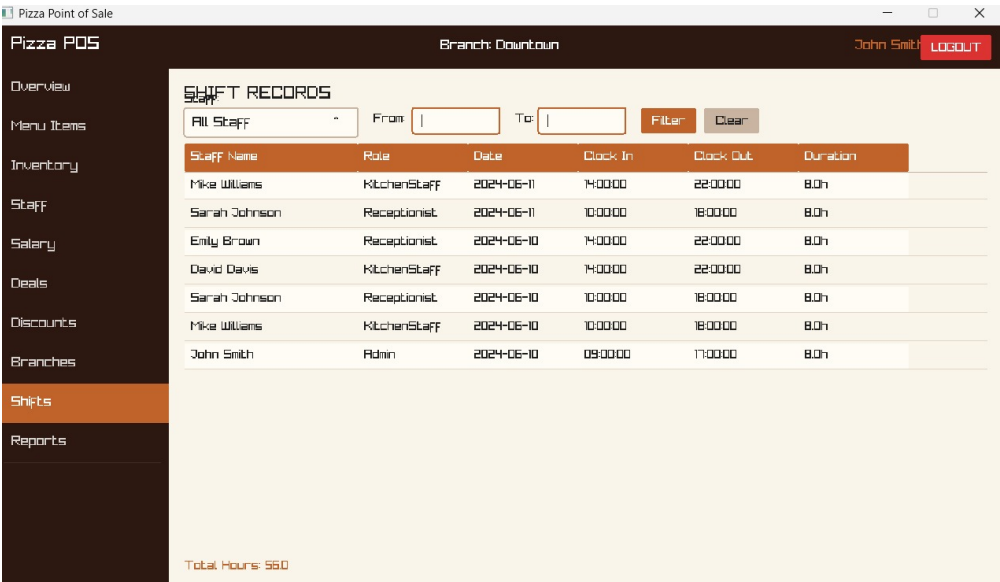


Figure 4.7: Shift Scheduling Panel – Admin View

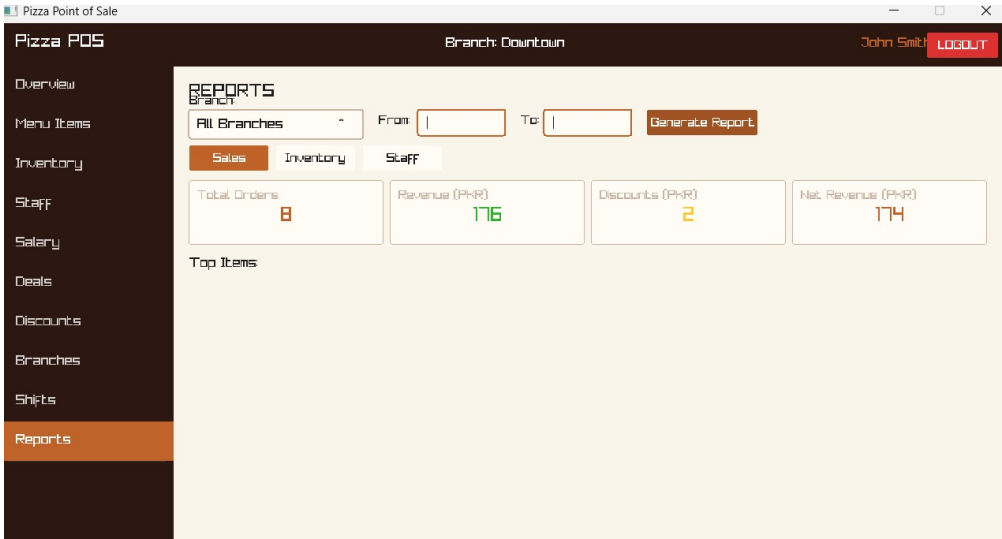


Figure 4.8: Sales Report Screen – Admin View

REFERENCES

- [1] Anthropic, “Claude AI Assistant,” <https://www.anthropic.com/claude>, accessed June 2026.
- [2] Overleaf, “Overleaf Online $\text{\LaTeX}$ Editor,” <https://www.overleaf.com>, accessed June 2026.
- [3] Lucid Software Inc., “Lucidchart,” <https://www.lucidchart.com>, accessed June 2026.
- [4] DeepSeek AI, “DeepSeek,” <https://www.deepseek.com>, accessed June 2026.

APPENDIX: AI USAGE AND PROMPTS

AI tools used: Claude (Anthropic), DeepSeek

Purpose: Learning database design concepts, exploring design choices, validating the ERD, generating L^AT_EX formatting, and producing sample data.

All AI-generated content was reviewed by every team member and checked against lecture notes before being included in this report. No content was copied without understanding it first.

Sr.	Prompt Used	Purpose
01	“Understand my database project. This ERD represents a POS and Inventory Management System using Chen’s notation.”	Initial project understanding
02	“I want to add a Price entity. How should it link to the menu item?”	Learning price history design
03	“I want an AddOn entity for extra cheese and toppings. How does it work with inventory?”	AddOn design pattern
04	“When a pizza with an add-on is ordered, does the quantity deduct from inventory?”	Inventory deduction logic
05	“Add a Shift entity for employees. What attributes should it have?”	Shift tracking design
06	“Add a Branch entity. Each branch has one admin and one admin manages many employees.”	Multi-branch architecture
07	“I want a Salary entity. Does it store just the current salary or all past payments?”	Salary history concept
08	“Give me a PlantUML code for the complete ERD in Crow’s Foot notation.”	ERD visualisation
09	“Should I add a Discount entity, or just a discount column in Order?”	Discount design pattern
10	“Explain the cardinality between Deal, DealItem, and MenuItem in simple terms.”	Relationship cardinalities
11	“Why is the relationship between Branch and Inventory optional-many?”	Optional cardinality understanding

12	“Why doesn’t MenuItem have an IsAvailable column? How do we handle per-branch availability?”	Per-branch availability design
13	“What business rules can you identify in my POS system from the ERD?”	Business rule documentation
14	“When a receptionist takes an order, how should the system assign it to kitchen staff automatically?”	Order assignment logic
15	“When an order is Pending, the employee can modify or cancel. When In Preparation, they cannot. How do I model this?”	Order status workflow
16	“Can a maximum of one discount apply to one order? How do I enforce that?”	Discount constraint design
17	“Only admin should see the salary table. How is this enforced?”	Role-based access control
18	“Explain 3NF to me with examples from a POS restaurant database.”	Normalisation concepts
19	“How should I handle derived attributes like Subtotal and ChangeDue in a normalised schema?”	Removing derived attributes
20	“Walk me through converting a 1:N relationship from an ERD into a relational schema.”	ERD-to-schema mapping
21	“Generate a LaTeX document structure for a university database design report.”	LaTeX report formatting
22	“Why should BranchId be added to the Order table? Is it necessary?”	Schema improvement
23	“Explain the relationship between Branch, BranchDeal, and Deal — what is optional and what is mandatory?”	Junction table cardinalities
24	“What is the purpose of the Shift entity? Can’t we just track hours in the Employee table?”	Shift entity justification
25	“Why do we add surrogate primary keys to junction tables like DealItem instead of using a composite key?”	Surrogate vs composite keys
26	“How do I connect MySQL 9.7 to a C++ application using the MySQL C API?”	C++ MySQL connectivity
27	“How do I include libmysql.dll and libmysql.lib in a VS Code C++ project?”	Build configuration

28	“Generate realistic sample data for a Pakistani pizza restaurant with three branches.”	DML test data
29	“Analyze this ERD. Does the design correctly enforce that if a deal item is unavailable, the whole deal is unavailable at that branch?”	Design validation
30	“Can you find any structural errors or missing relationships in this ERD?”	ERD error checking
31	“You are building a C++ POS system for a multi-branch pizza shop using Raylib and MySQL C API on Windows MinGW. CRITICAL: mysql.h and raylib.h must NEVER appear in the same file. models.h must include NEITHER. Generate the file src/models/models.h with all required structs: Branch, Employee, Salary, Shift, Category, MenuItem, Price, Inventory, MenuIngredient, BranchMenuItem, Discount, Order, OrderItem, Deal, DealItem, BranchDeal, CartItem, LowStockAlert.”	Generating models.h header file with all data structs for the POS system
32	“Master Prompt — Raylib + MySQL C++ Project on Windows MinGW w64devkit. Compiler at C:\raylib\w64devkit. Raylib headers and libs in w64devkit. MySQL Server 9.7 headers copied into project include/ folder. libmysql.lib and libmysql.dll copied to project root. Critical rules: (1) Never mix mysql.h and raylib.h in the same file due to Windows header conflicts with Raylib’s Rectangle, DrawText, CloseWindow, ShowCursor, PlaySound, and LoadImage. (2) Separate MySQL into db.h (declarations only, no mysql.h) and db.cpp (all MySQL logic). main.cpp includes only raylib.h and db.h. (3) Always use MySQL C API (mysql.h + libmysql) with MinGW — never use mysql_driver.h or mysql_connection.h. Compile command: g++ with -lraylib -lmysql -lopengl32 -lgdi32 -lwinmm -lws2_32. Run with game.exe not main.exe.”	Master project setup and architecture rules for Raylib + MySQL on MinGW